Intermediate (apply the follow-up template yourself) — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `followup_template.md`: interviews are genuinely won or lost in the follow-up questions, not the first answer you give. This exercise answers a scenario and then asks yourself the standing follow-ups that always come next, before a real interviewer ever gets the chance to. **If not:** every "why not X instead?" would arrive as a surprise you have to improvise through, live.

Write your own answer first, in `practice/followup_template.md`, then compare.

Scenario question: "Your RAG agent's retriever is returning irrelevant chunks in production, but it looked fine in testing. Walk me through how you'd debug it."

Solution

Story: this keeps the same order of follow-ups, but now every one names a specific, checkable mechanism rather than a vague category, and "what if it fails again" turns into an actual concrete design fix, not just an acknowledgment. **If not:** your answers would stay stuck at the category level, and a strong interviewer would keep pressing with "how, specifically?" until you ran out of depth.

First answer: "I'd isolate the layer first — pull the actual retrieved chunks for a handful of the bad live queries and check relevance directly, before assuming it's the retriever at all, since a prompt or generation regression can look identical to a retrieval regression from the outside."

Follow-up — what's underneath: "Assuming the chunks themselves are genuinely off-topic, the three usual suspects are: index staleness — the underlying documents changed and weren't re-embedded; a chunking regression — a recent change to chunk size or boundaries is now splitting semantic units mid-sentence, hurting embedding quality; or distribution shift — the live query distribution has drifted from what the retriever was tuned and tested against."

Follow-up — how would you debug it: "I'd re-run the original test suite's queries against the live index first — if those still score well, the retriever itself is fine and the problem is upstream (new query patterns we never tested). If the suite now fails too, I'd bisect: check the index's last update timestamp against the last chunking-logic deploy, since that pinpoints which of the two most likely causes it is."

Follow-up — what happens if it fails again: "Right now this fails silently — a bad retrieval still produces a fluent, plausible-sounding wrong answer, which is the worst kind of failure because nothing looks broken. I'd add a similarity-score threshold: below it, the system says it's not confident instead of generating from weak context."

Why this answer works: it's this document's Normal and Deep rungs, done properly, stacked on top of the standing follow-up list — it starts by isolating the failing layer instead of assuming the premise ("the retriever") is even correct, which is a senior instinct, not a junior one. Each follow-up answer names a *specific, checkable mechanism* (index staleness, chunking regression, distribution shift) instead of a vague category. It treats "what happens if it fails again" as a design gap to fix (a confidence threshold), not just a fact to report.